SideQuest:
Gamified Local Community & Gig Platform

Fundamentals of Software Engineering

Date of Submission: May 3, 2026

Group Members:

- Talmeez ur Rehman (Roll Number: 24L-2576) - Project Lead & Frontend UI/UX Developer
- Muhammad Yaseen (Roll Number: 24L-2515) - Backend Developer & Software Architect
- Taimur Amir (Roll Number: 24L-2518) - Requirement Engineer & QA Tester

Project Introduction

Project Title: SideQuest: Gamified Local Community & Gig Platform

Introduction: SideQuest is a full-stack, hybrid community-building and gig-economy web application designed to gamify daily tasks and social needs into localized "Quests." The platform was developed incrementally across three Agile sprints:

- **Sprint 1 (Foundations):** Established the core SQL database, JWT-based authentication, and the primary interactive Notice Board where users can post, filter, and bookmark local bounties.
- **Sprint 2 (The Gig Economy):** Transformed the board into a functional economy by introducing a strict Quest Application system, secure Escrow Wallets, and a mutual completion lifecycle to guarantee scam-free gold transfers.
- **Sprint 3 (The End-Game Polish):** Solidified the community with Grandmaster (Admin) moderation tools, Tier 1 KYC CNIC verification, real-time quest-isolated Tavern Chat, custom Avatars, and a Global Hall of Fame leaderboard.

User Stories (All Sprints)

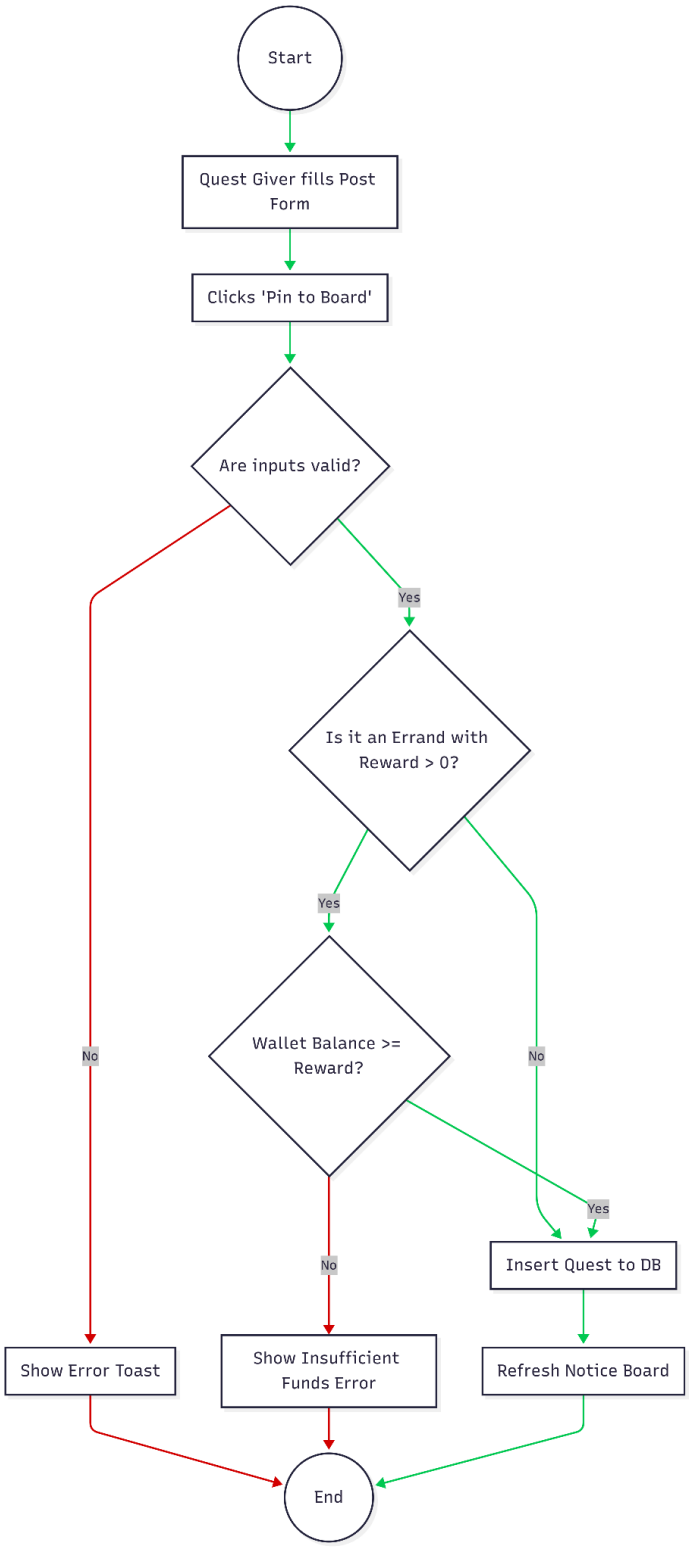
Story ID	Title	Priority	Description	Acceptance Criteria
US-01	The Quest Board	High	As an adventurer, I want to view and filter a board of available quests so I can find local tasks.	Quests fetch on load; Displayed as cards; Dynamic filtering without page reloads.
US-02	Posting a Bounty	High	As a quest-giver, I want to post new quests so adventurers can offer help.	Form validation; "Social" quests force 0 reward; Board auto-refreshes on post.

US-03	Bookmarking Quests	Medium	As a user, I want to bookmark quests to easily track them later.	Toggle visual state; Saves to DB; "Saved" tab filters correctly.
US-04	Guild Authentication	Critical	As a new user, I want to securely register and log in to ensure a safe community.	CNIC regex validation; bcrypt password hashing; Protected routes.
US-05	Quest Application	Critical	As an adventurer, I want to apply for a quest so the Giver knows I am willing to do it.	Users cannot apply to own quests; Approving one applicant rejects all others.
US-06	Secure Escrow	High	As a user, I want funds held in Escrow to guarantee payment and prevent scams.	Verify Giver has funds; Deduct from Wallet and hold in Escrow balance.
US-07	Status Lifecycle	High	As a user, I want to track the status lifecycle (Open > Accepted > InProgress > Completed).	Taker initiates "Start"; Mutual "Complete" flags required to release funds.
US-08	KYC Verification	Critical	As an adventurer, I want to upload my CNIC to earn a Tier 1	Image compression; Admin dashboard for approval/reject

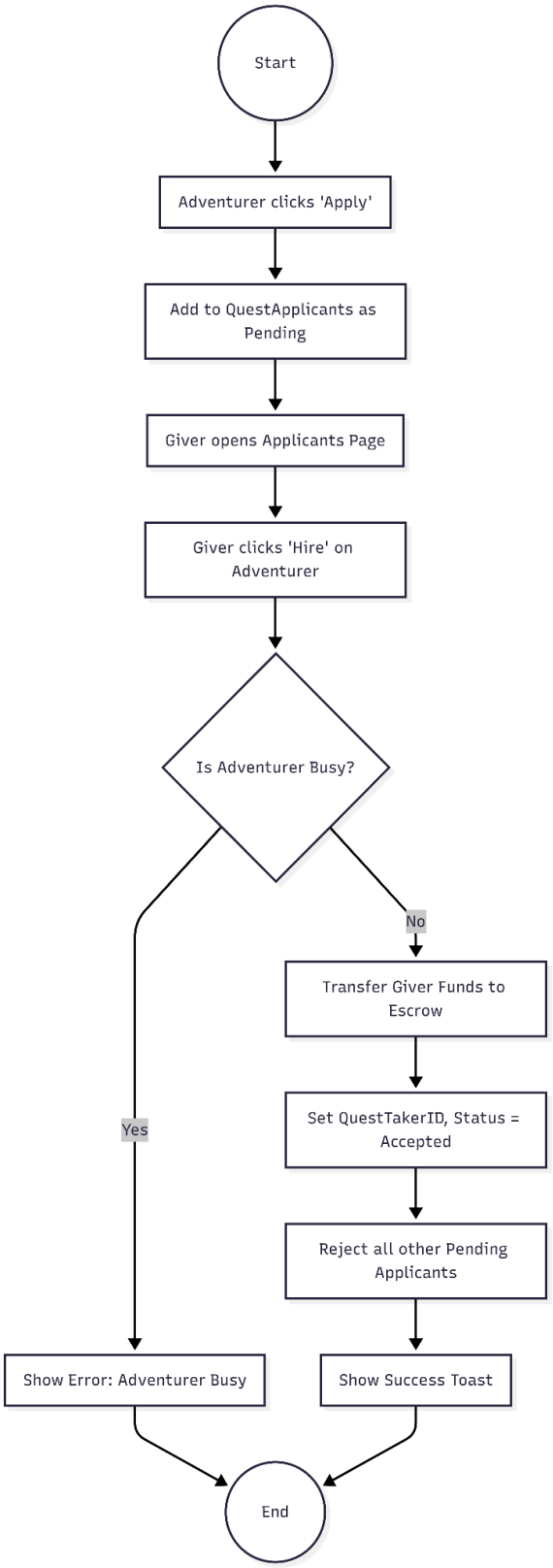
			verification badge.	ion/revocation.
US-09	Tavern Chat	High	As a hired user, I want to direct messages to my partner to coordinate the gig.	Isolated by QuestID/Users; Real-time polling; Hidden upon quest completion.
US-10	Hall of Fame & Avatars	Medium	As a user, I want to upload an avatar and view the top-rated adventurers.	Ranks top 10 by Rating/Quests; Safely falls back to default avatar icon.
US-11	Admin Moderation	High	As an Admin, I want to wield a 'Ban Hammer' and view platform statistics.	Admins can toggle account suspension; Suspended users receive 403 Forbidden.

Design

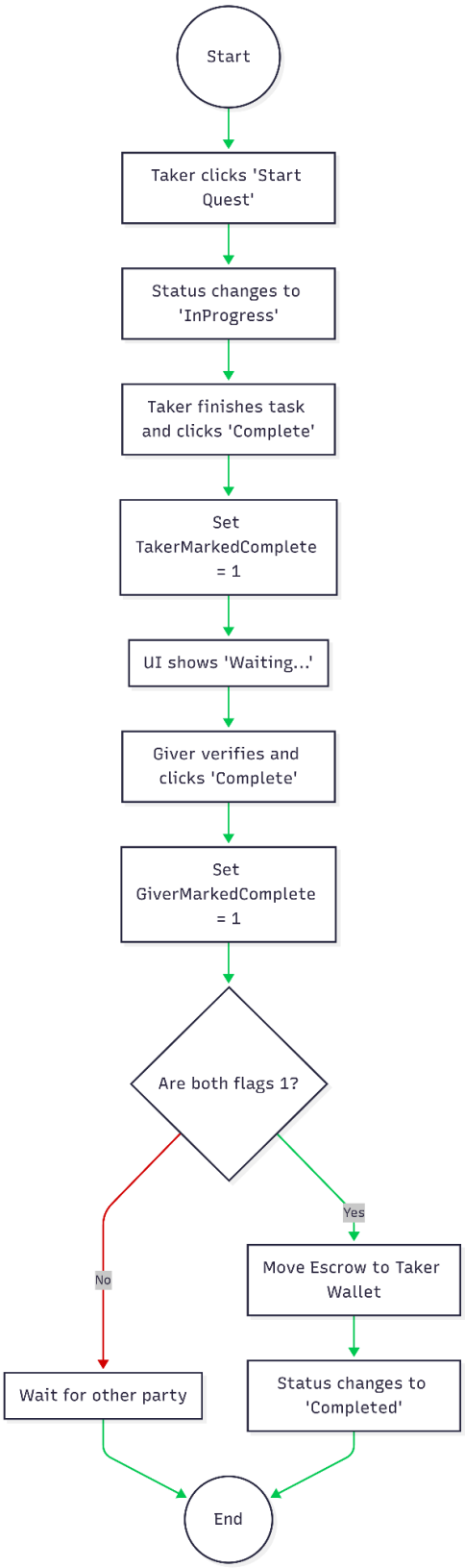
- **Activity Diagram - Post A Bounty:**



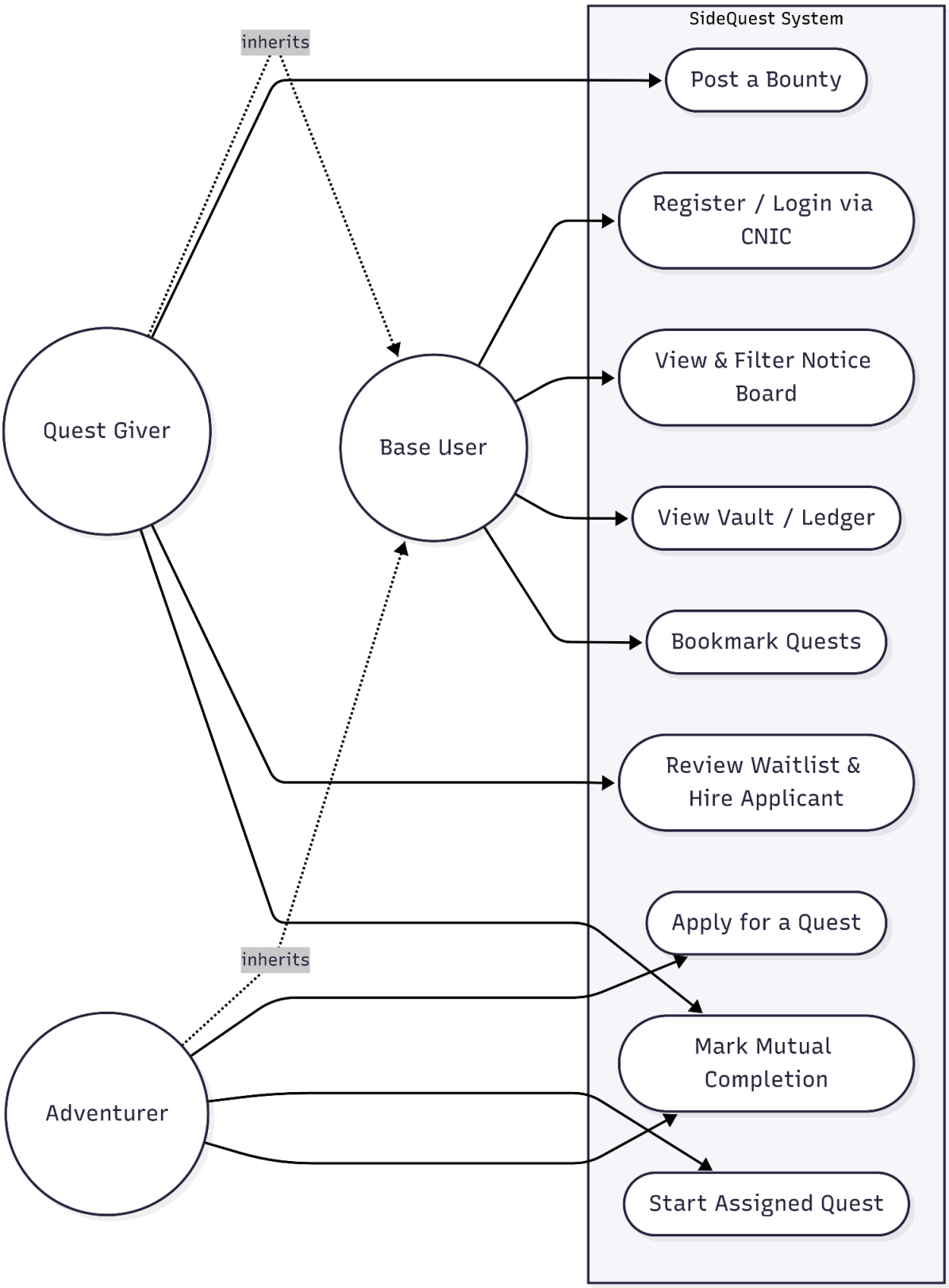
- **Activity Diagram - The Hiring Process:**



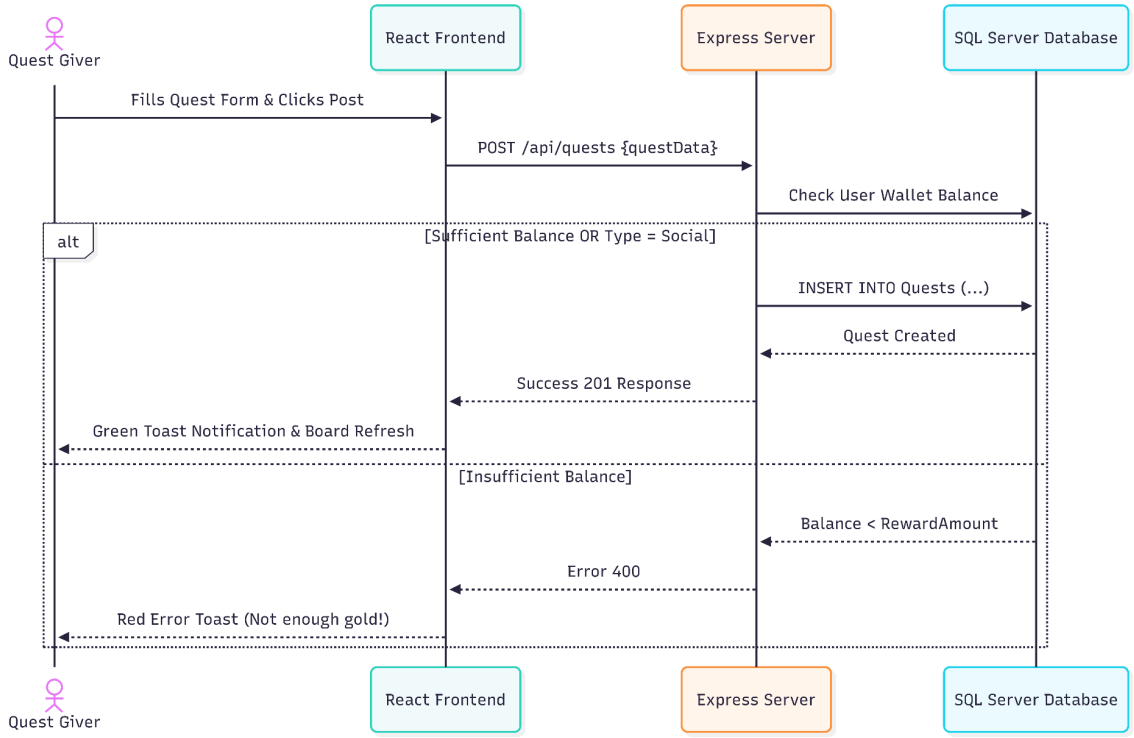
- **Activity Diagram - Mutual Completion:**



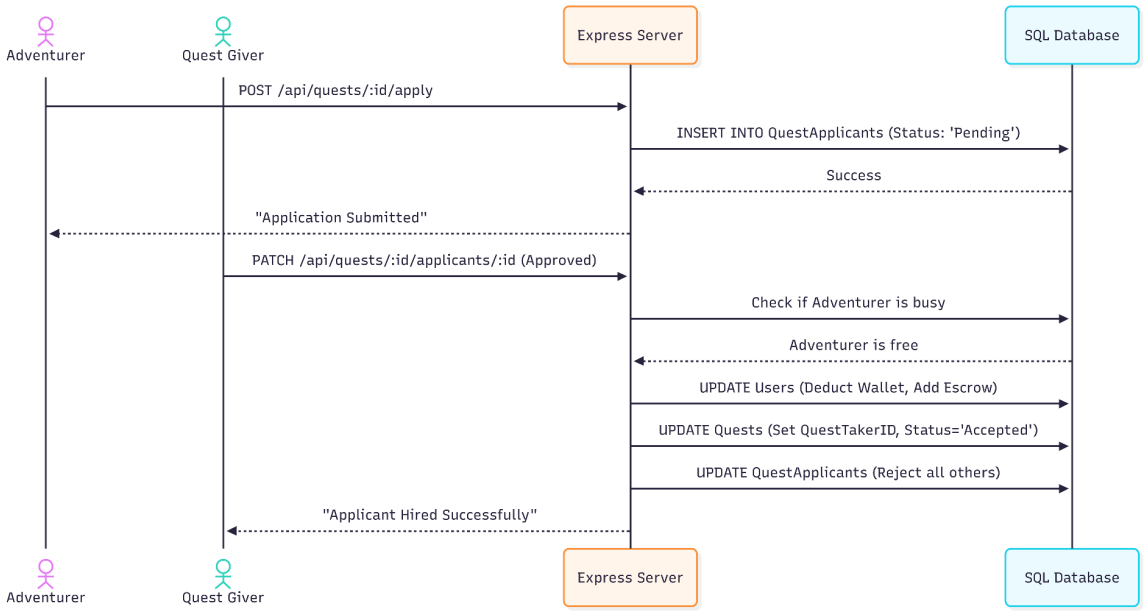
- **Use Case Diagram:**



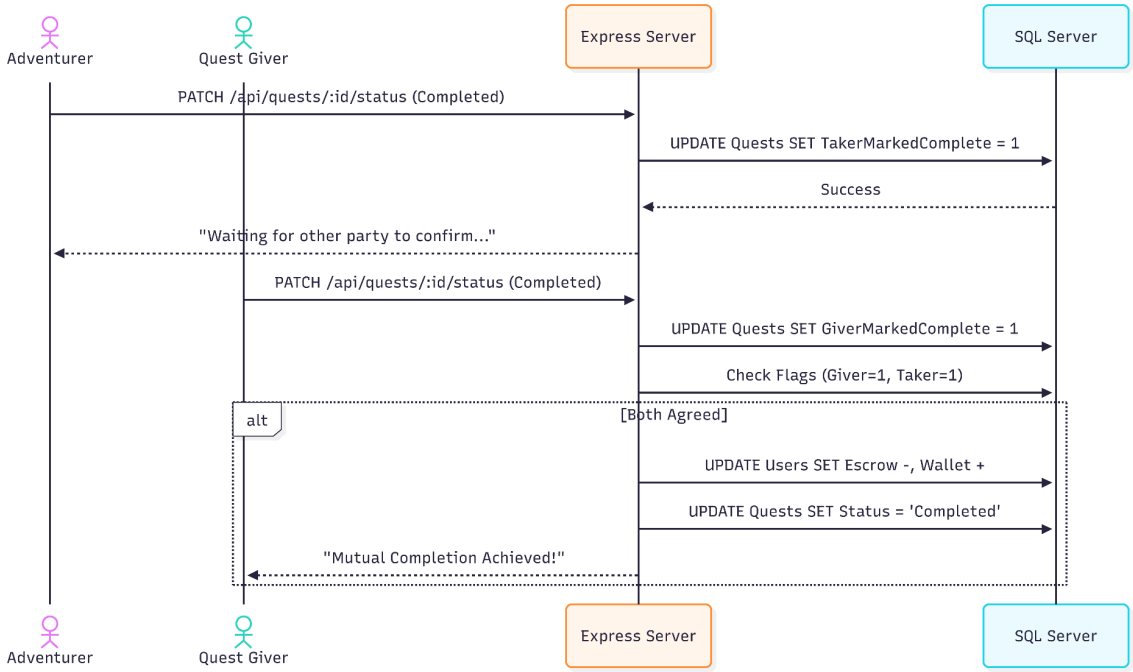
- ## Sequence Diagram - Post A Bounty:



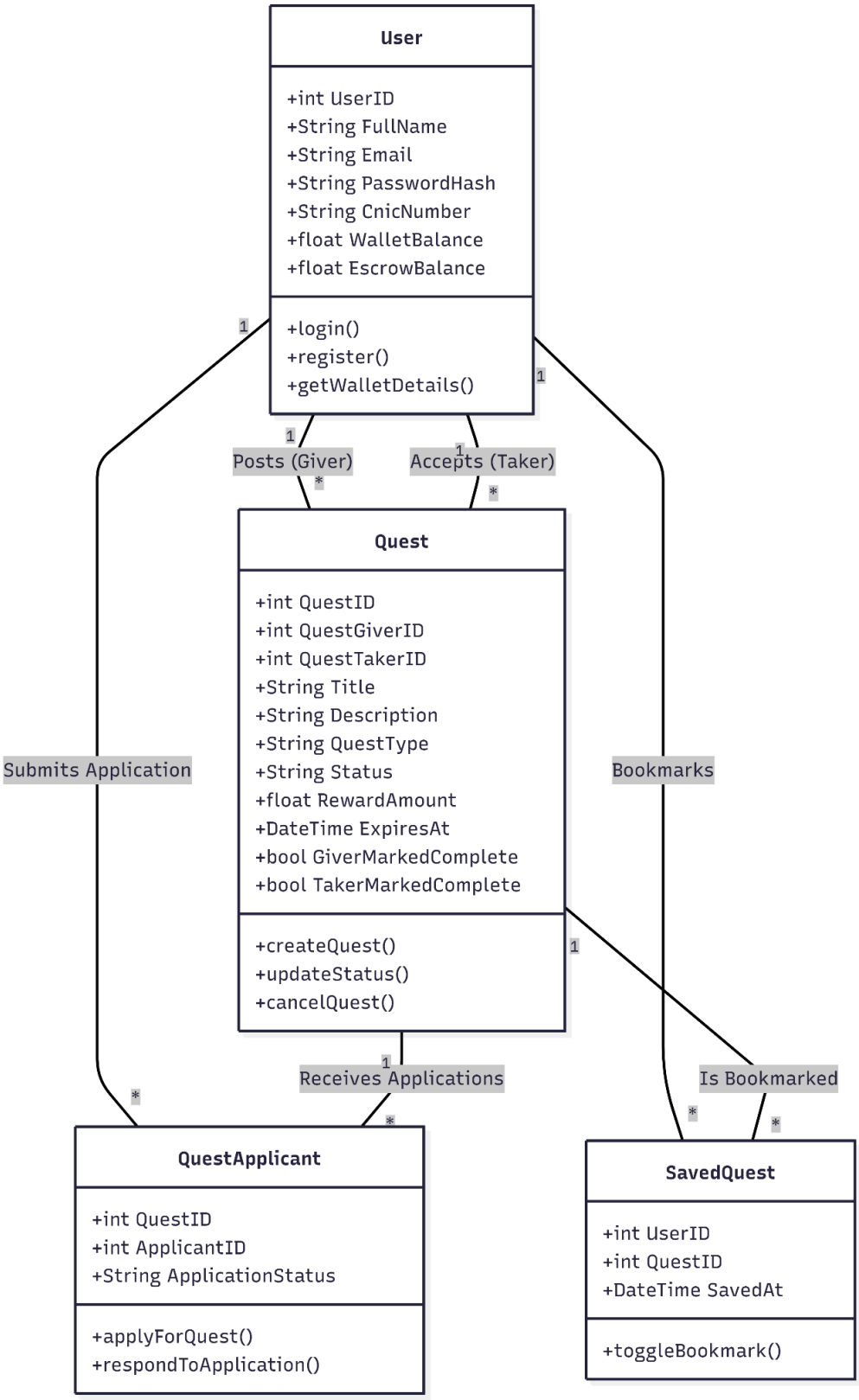
- ## Sequence Diagram - The Hiring Process:



- **Sequence Diagram - Mutual Completion:**



- **Class Diagram:**

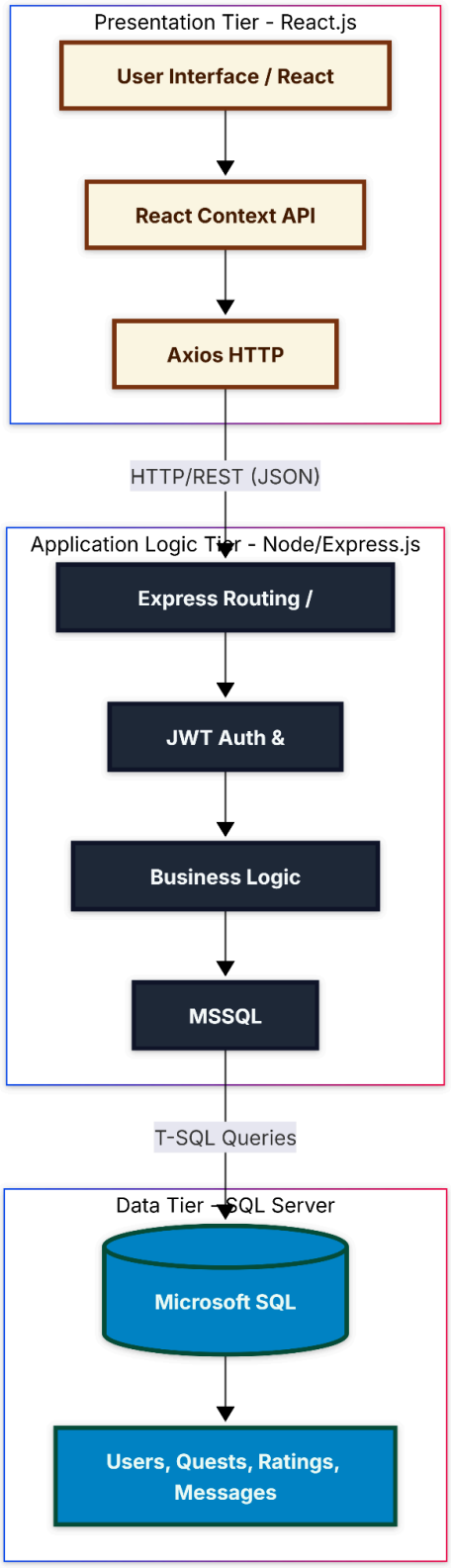


Architecture

SideQuest utilizes a **3-Tier Architecture** based on the MERN stack paradigm, substituted with Microsoft SQL Server for robust relational data constraints.

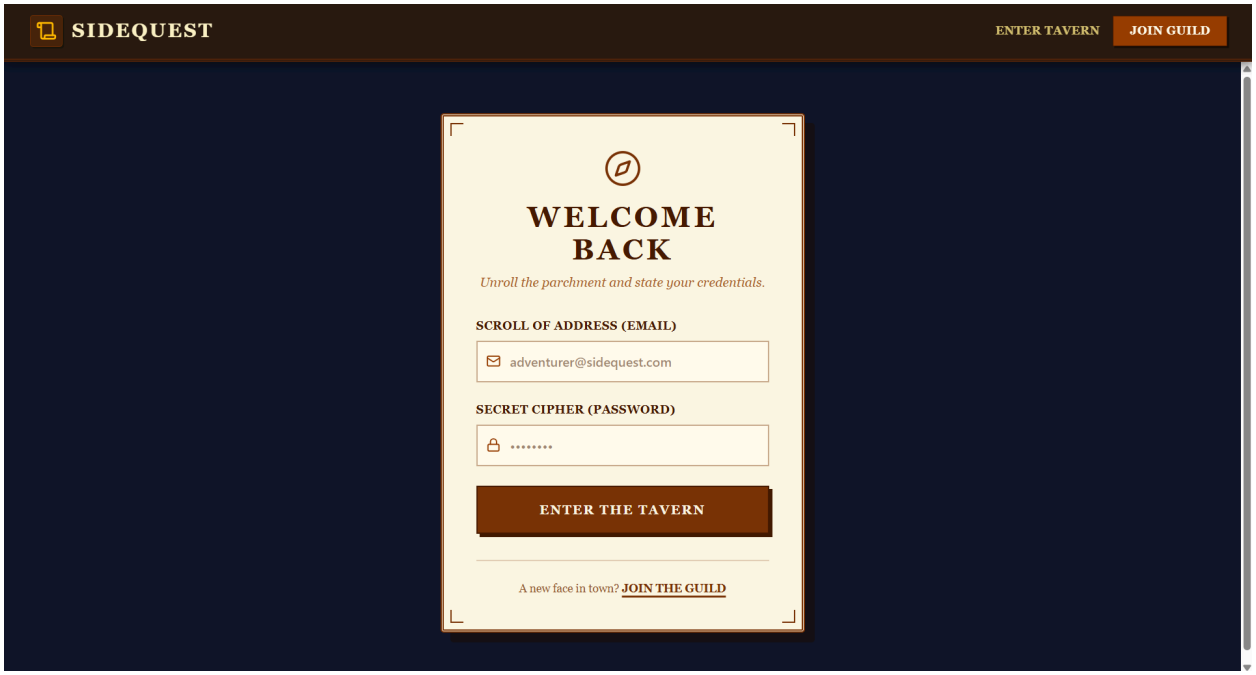
1. **Presentation Tier (Frontend):** Built with React.js and Tailwind CSS. Uses Context API for global state management and Axios for HTTP communications.
2. **Logic Tier (Backend):** Built with Node.js and Express.js. Handles business logic, JWT-based security middleware (protect, adminOnly), file parsing, and RESTful routing.
3. **Data Tier (Database):** Hosted on MSSQL. Enforces data integrity through Foreign Keys, CHECK constraints, composite keys (Saved Quests), and transactional math (Escrow logic).

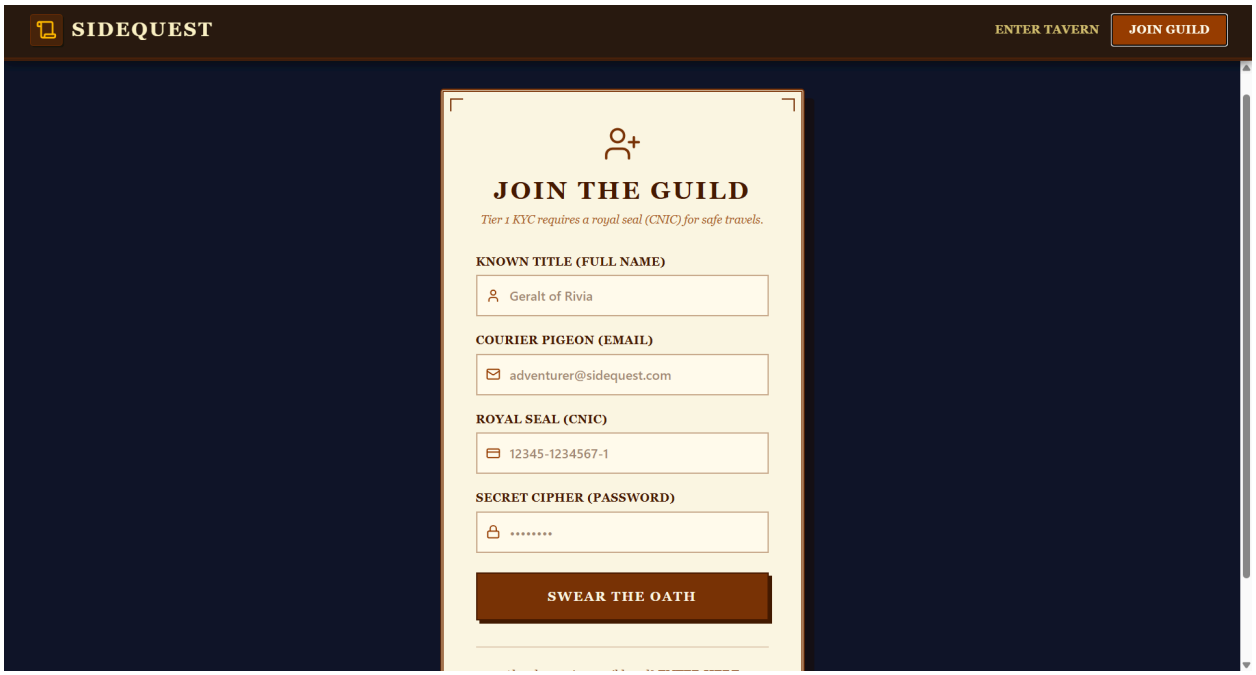
- **Architectural Diagram:**



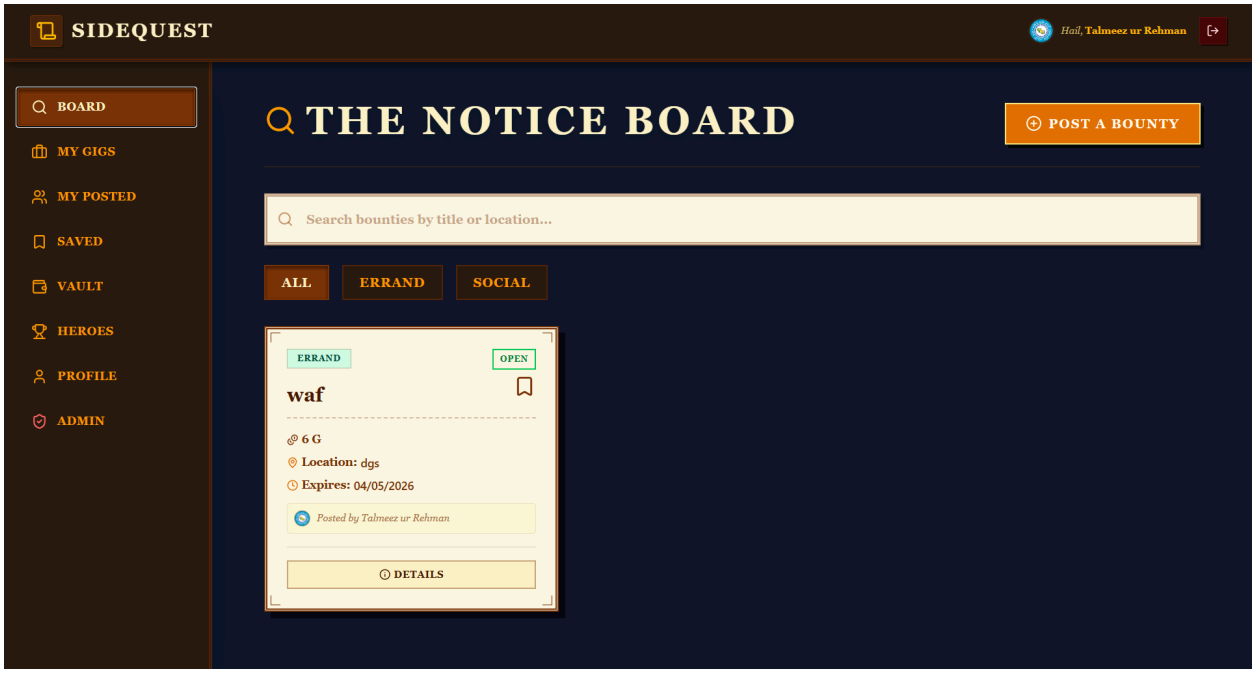
Implementation Screenshots

- **Login and Registration Portals:**

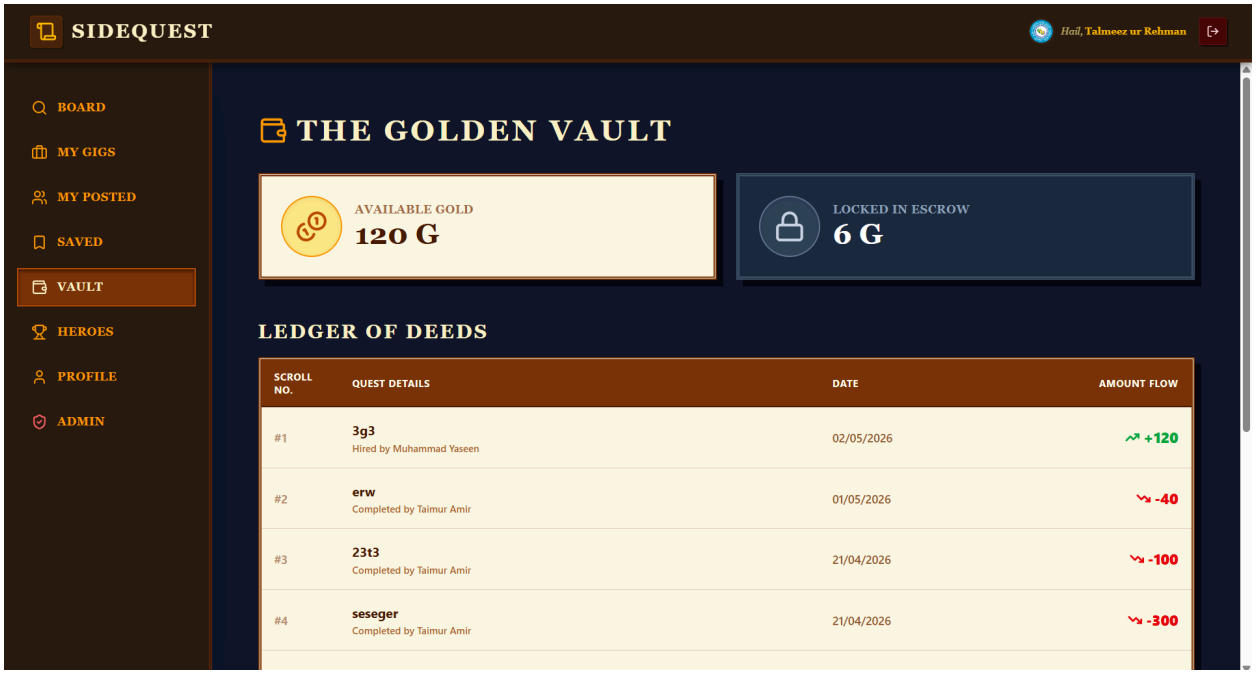




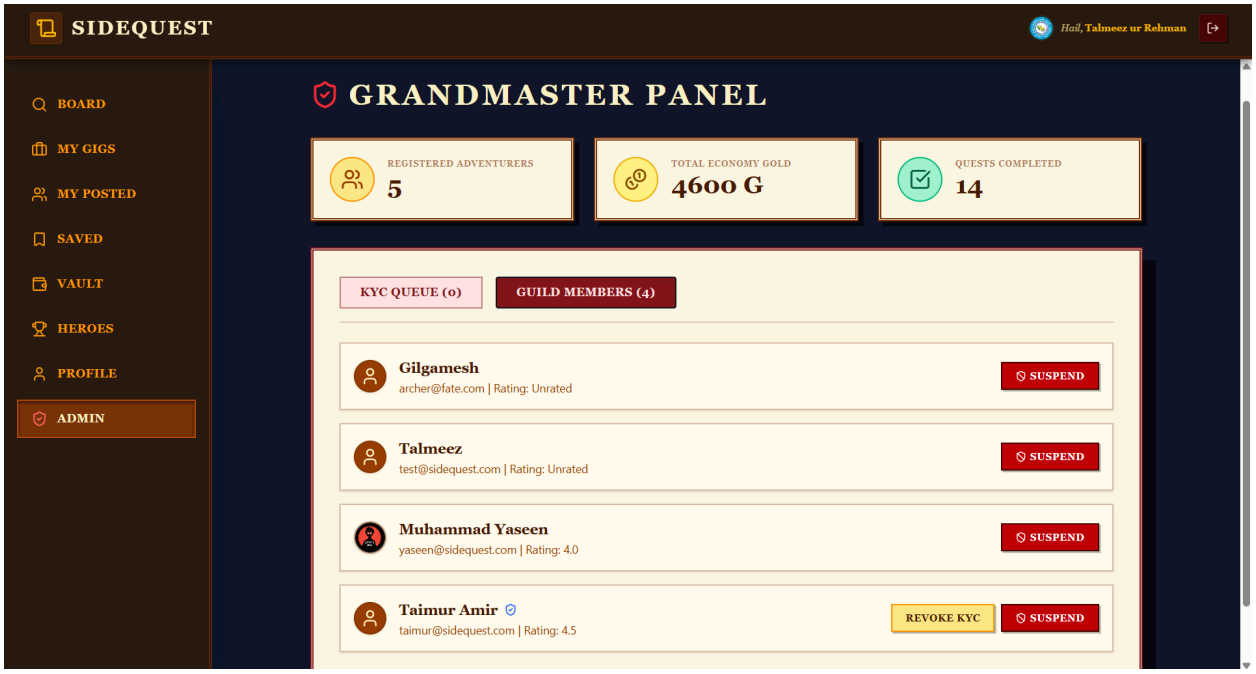
- The Main Notice Board with Search & Sidebar:



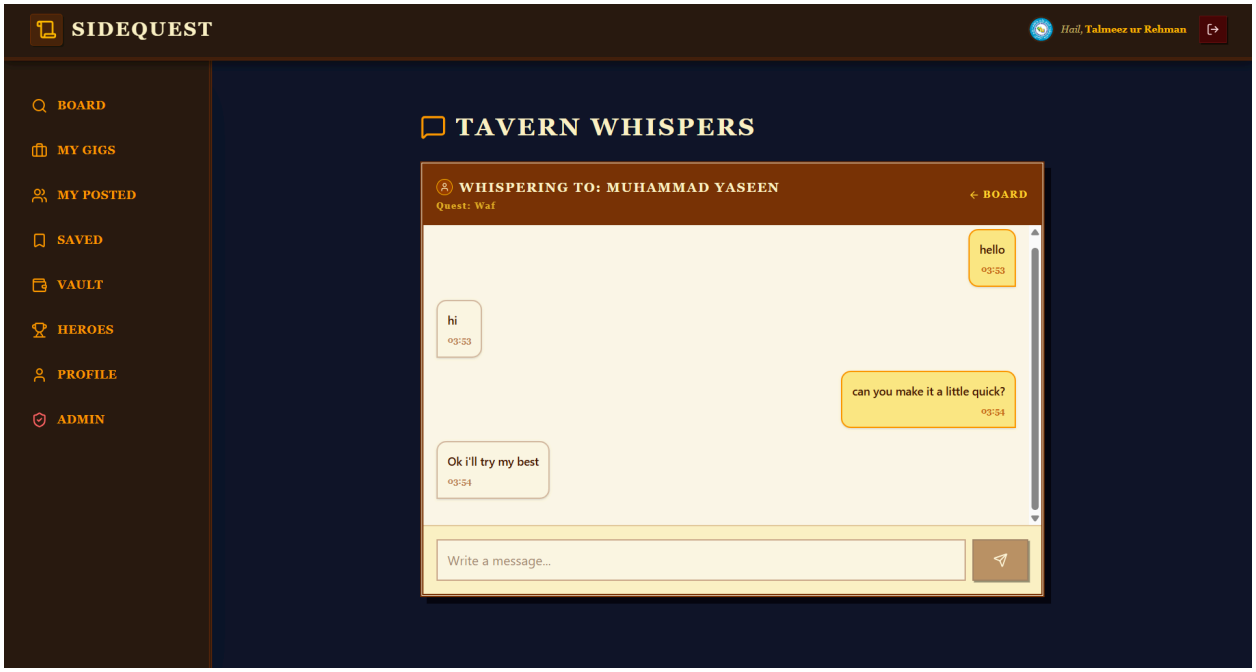
- The Golden Vault:



- **Grandmaster Admin Panel:**

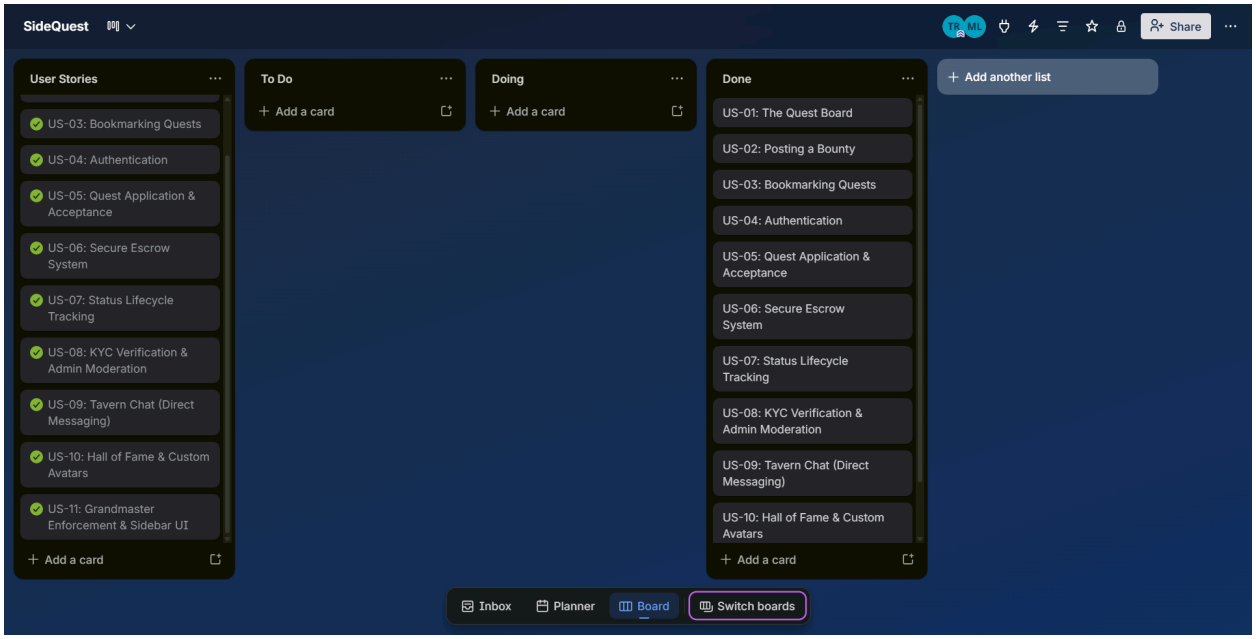


- **Real-time Tavern Chat UI:**



Product Burn-down Chart for the Project

Trello Board Screen Shots



Boundary Value Analysis Testing

1. Login Interface

1. LOGIN INTERFACE BOUNDARY VALUE ANALYSIS

Testing boundaries for Email and Password inputs.

TEST CASE NO	TEST CONDITION	INPUT: EMAIL	INPUT: PASSWORD	EXPECTED RESULT	ACTUAL RESULT	STATUS
01	Invalid Email + Valid Password	witcher.sidequest	secure123	HTML5 prevents submission (invalid format).	UI blocks submission.	✅
02	Valid Email + Below Min Password	witcher@sidequest.com	abcde	System rejects auth due to short password.	UI shows password error.	✅
03	Both Minimum Valid Boundaries	a@b.c	abcdef	System accepts lowest valid limit and attempts auth.	System attempts login.	✅
04	Max Valid Email + Max Valid Password	a * 245 + @gmail.com	a * 50	System handles extremely long boundaries safely.	System attempts login.	✅
05	**Both Invalid Boundaries**	witcher@.com	12345	System triggers multiple validation errors; blocks.	Form halted by UI.	✅

2. Registration Interface

2. REGISTRATION INTERFACE BOUNDARY VALUE ANALYSIS

Testing boundaries for Name, Email, Password, and CNIC inputs.

TEST CASE NO	TEST CONDITION	NAME	EMAIL	PASSWORD	CNIC	EXPECTED	ACTUAL	STATUS
01	All Valid Inputs (Happy Path)	Geralt	g@rivia.co	roach123	35202-1234567-1	System validates all fields and registers user.	Registration successful.	✅
02	Invalid Name + Valid Others	Ge	g@rivia.co	roach123	35202-1234567-1	Rejects submission due to short Name length.	UI/Backend rejects.	✅
03	Invalid Email Format + Valid Others	Geralt	g.rivia.co	roach123	35202-1234567-1	HTML5 intercepts invalid email format.	UI prevents submission.	✅
04	Below Min Password + Valid Others	Geralt	g@rivia.co	roach	35202-1234567-1	Catches short password error and rejects.	Shows password error.	✅
05	Valid Others + Invalid CNIC Chars	Geralt	g@rivia.co	roach123	35202-ABCDEFG-1	UI strips alphabetic characters immediately.	Letters stripped.	✅
06	Valid Others + Max Length CNIC bounds	Geralt	g@rivia.co	roach123	35202-1234567-12	UI prevents typing the 16th character.	Stops at 15 chars.	✅
07	Pure Numbers CNIC + Valid Others	Geralt	g@rivia.co	roach123	3520212345671	UI auto-formats the raw 13 numbers with dashes.	Auto-formats to valid.	✅
08	Max Valid Stress Test (All maximums)	A * 100	a * 245 + @g.c	A * 50	35202-1234567-1	System handles extremely long valid strings safely.	Registration successful.	✅
09	**Total Chaos (All Invalid)**	(Empty)	bademail	123	123-abc-!!!	Multiple validation triggers hit simultaneously.	Form halted by UI.	✅

Work Division Between Group Members

Talmeez ur Rehman (Project Lead & Frontend UI/UX Developer):

- Designed and built the entire React.js frontend and responsive Tailwind CSS layouts.
- Engineered the dynamic Sidebar Dashboard, Quest Details Modals, and the Golden Vault Ledger UI.
- Implemented client-side HTML5 Canvas algorithms to compress Base64 KYC and Avatar images.
- Built the real-time Tavern Chat UI, the magical Search Bar filter logic, and completely replaced native browser alerts with custom z-indexed React modal overlays.
- Managed overall project direction and UI/UX cohesion.

Muhammad Yaseen (Backend Developer & Software Architect):

- Architected the entire Node.js/Express backend and the Microsoft SQL Server database schema.
- Developed RESTful API endpoints for Authentication, Quest Management, and Wallet Ledgers.
- Engineered robust, ISNULL math logic to handle secure Escrow locking and refunds.
- Developed the Admin controllers, UTC timezone parsers, complex UNION queries for the Hall of Fame, and the dual-flag mutual completion patching system.
- Implemented JWT protection and adminOnly security middleware.

Taimur Amir (Requirement Engineer & QA Tester):

- Handled Requirement Engineering, defining precise acceptance criteria for all User Stories.
- Executed rigorous Boundary Value Analysis (BVA) and Edge-Case Testing (e.g., catching overlapping application "Greedy Goblin" bugs and SQL serialization buffer illusions).
- Managed Agile operations, maintained the Scrum/Trello boards, and generated the Burn-down charts.
- Drafted and formatted all iteration documentation, ensuring traceability from User Stories to Structured Specifications.

Lessons Learnt by Group

- **The Complexity of Financial State:** We learned that building a gig economy requires extreme precision with database mathematics. Handling Escrow states taught us the importance of accounting for edge cases like quest cancellations, preventing duplicate applications, and ensuring dual-party consent to release funds.
- **Frontend vs. Backend Data Serialization:** During Sprint 3, we discovered that SQL BIT fields can occasionally serialize into raw Buffers ({type: "Buffer", data: [1]}) when passed to a JavaScript frontend. Learning to proactively decode and sanitize API payloads on the frontend was a major lesson in data integrity.
- **Asynchronous UX is Crucial:** We learned that relying on native alert() boxes breaks application flow. Transitioning to custom React context (Global Toasts) and Z-indexed

Modals drastically improved the professional feel of the application.

- **Team Synergy and Version Control:** Working concurrently on complex overlapping features (like Chat, Authentication, and the Quest Board) taught us the immense value of clear component architecture and modular route design, preventing massive merge conflicts and tightly coupling our efforts to the Scrum board.